CSE235 Week 4: While and For Loops

This week, we'll be discussing two more conditional statements in Python, the while and for loops. While loops execute a block of code so long as its condition is true. For loops iterate over a range of values.

While Loops

A while loop is used to execute a set of statements *while* a given condition is true. Below is the basic syntax:

```
while condition:
    statement
```

The while loop starts with the `while` keyword followed by a condition. Just like the if statement, the condition does not require parentheses, and is followed by a colon. The body of the while loop is a block, so the statements that are executed by the while loop must be indented. Below is a basic while loop that reads and sums five numbers:

```
# initialize sum and counter variable to 0
sum = 0
i = 0
# run the loop as long as i is less than 5
while i < 5:
    # add the number input by the user to the sum
    sum += float(input("Enter a number: "))
    # increase i by 1
    # the loop will run when i is 0, 1, 2, 3, and 4
    # when i is increased to 5, the loop will end
    i += 1

# the loop is over, so return to the original indent level
print("Total sum =", sum)
```

This is an example of a definite loop. Definite are controlled by a counter that is compared to a value that is either known when writing

the code, like the example above, or read in from the user before the loop. Below is an example that will ask the user how many numbers they will enter, then computes the average of the numbers after they are entered:

```
# read in the total number of numbers from the user
total = int(input("Enter how many numbers will be input: "))

# initialize counter and sum
i = 0
sum = 0

# run the loop while i is less than the total
while i < total:
    # add the number input by the user to the sum
    sum += float(input("Enter a number: "))
    # increase i by 1
    i += 1

# compute the average
average = sum / total
# print the average
print("The average is", average)
```

Counter controlled loops do not have to count up like the previous examples. You can also count down:

```
# reading in the drink choice
drink = input("Enter drink: ")
# reading in the number of bottles
bottles = int(input("Enter number of bottles: "))
# line that separates the input from the song
print("\n-----------------------------------------------\n")

# while loop that prints the song
# since we are counting down, we want our control variable to be
# greater than the minimum, which is 0 in this case
while bottles > 0:
    # print the first set of lyrics in a verse
    print(bottles, "bottles of", drink, "on the wall")
    print(bottles, "bottles of", drink)
    print("Take one down, pass it around")
    # since the next line of the song has 1 fewer bottle
    # we'll decrement that value of bottles here.
    bottles -= 1
    print(bottles, "bottles of", drink, "on the wall")
```

The other type of condition you can have for a while loop is indefinite, also called sentinel controlled. In these loops, the value entered by the user is compared to a "sentinel value". A sentinel value is a value that is predefined or read from the user before the loop to determine when the loop stops. The loop will then run until the user enters that value. Below is an example that reads names from the user until they enter the value "quit", at which point the loop ends:

```
# read a name from the user
name = input("Enter a name (or 'quit' to stop): ")

# run the loop while name is not "quit"
while name != "quit":
    print("You entered:", name)
    # read another name
    name = input("Enter a name (or 'quit' to stop): ")

# remember to return the the old indent level when the loop is done
print("Loop is done")
```

We can also read the sentinel value from the user. This program shows how to compute the average of numbers input by the user using a sentinel controlled loop:

```
# read in the number to stop at
stop = float(input("Enter the STOP number: "))
# initialize the count and sum to 0
count = 0
sum = 0

# read in a number
num = float(input("Enter number: "))

# loop while the input number is not the stop value
while num != stop:
    # add the number that was just input to the sum
    sum += num
    # increase the count
    count += 1

average = sum / count
print("The average is", average)
```

For Loops

For loops in Python are a little different compared to C-like languages (C, C++, C#, Java, etc.). In those languages, a for loop consists of a variable declaration, condition, and step value. In Python, a for loop looks like the following:

```
for variable in container:
    statement
```

All for loops in Python are range based, meaning they iterate through a collection of values. If you want to use a Python for loop like a traditional for loop, there is the `range()` function that is used to generate a sequence of integers to iterate through. There are three versions of `range()`:

- `range(end)` - this version returns integers from 0 to one less than `end`, increasing by 1 each time
- `range(start, end)` - returns integers from `start` to one less than `end`, increasing by 1 each time
- `range(start, end, step)` -returns integers from `start` to one less than `end`, increasing by `step` each time. This version can also be used to count backwards

Here is the first while loop example rewritten using a for loop:

```
sum = 0

# loop will run for i = 0, 1, 2, 3, 4
for i in range(5):
    sum += float(input("Enter a number: "))

print("Total sum =", sum)
```

A for loop can be used anywhere a definite while loop is used:

```
total = int(input("Enter how many numbers will be input: "))
sum = 0

# a variable can be used for the max as well
for i in range(0, total):
    sum += float(input("Enter a number: "))
```

```
average = sum / total
print("The average is", average)
```

And the third example, showing how `range()` can be used to count down:

```
# reading in the drink choice
drink = input("Enter drink: ")
# reading in the number of bottles
bottles = int(input("Enter number of bottles: "))
# line that separates the input from the song
print("\n-----------------------------------------------\n")

# using a for loop to count down.
# to count down, pass a negative as the step value
#  and have the start be larger than the end of the range
for bottle in range(bottle, 0, -1):
    # print the first set of lyrics in a verse
    print(bottle, "bottles of", drink, "on the wall")
    print(bottle, "bottles of", drink)
    print("Take one down, pass it around")
    # the next line of the song has 1 fewer bottle
    print(bottle - 1, "bottles of", drink, "on the wall")
```

For loops cannot be used in place of an indefinite loop.

Break and Continue Statements

Break statements are used to exit a loop early. A break statement is simply the keyword `break` on its own line. Continue statements skip to the next iteration of a loop. A continue statement consists of the `continue` keyword on its own line. Below is an example of continue and break being used:

```
# prints only odd numbers
for i in range(10):
    if i % 2 == 0:
        # skip to the next loop iteration
        continue
    print("i =", i)

# breaks out of the while loop early
# if the user enters a negative number
count = 0
MAX = 10
```

```
while count < MAX:
    number = float(input("Enter a positive number: "))
    # check if the number is less than 0
    if number < 0:
        print("Negative entered.")
        # break out of the loop
        break
    print("You entered", number)

print("After the loop")
```

Infinite Loops

An infinite loop is a loop whose condition will never be false. Sometimes, this is done intentionally, and a break statement is used to break the loop. In general, though, infinite loops are the result of programmer mistakes. Here are some common errors that cause infinite loops:

- Using the wrong comparison
- Incrementing a counter when you should decrement (and vice versa)
- Using an impossible sentinel value

Examples

Here are some examples using while and for loops:

Even Numbers from 0 to 100 (While Loop)

```
# start count at 0
i = 0
# go up to 100, including 100
while i <= 100:
    print(i)
    # increase by 2 to only print even numbers
    i += 2
```

Even Numbers from 0 to 100 (For Loop)

```
# go up to 100, including 100
# use the step of 2 to only have even numbers
for i in range(0, 101, 2):
    print(i)
```

Printing a Rectangle (For Loop)

```
# outer loop handles the rows
for i in range(10):
    # inner loop prints each row of 10 '#'
    for j in range(10):
        print('#', end='')
    # move to the next line
    print()
```

The output would look like this:

```
##########
##########
##########
##########
##########
##########
##########
##########
##########
##########
```

Grade Summation (While Loops)

```
# read first name
name = input("Enter first student: ")

# run loop while name is not "quit"
while name != "quit":
    # initialize variables for computing grades
    count = 0
    grade = 0
    total_grade = 0

    # read first grade
    grade = float(input("Enter first grade for", name, ": "))

    # read grades while grade is >= 0
    while grade >= 0:
        count += 1
        total_grade += grade
        grade = float(input("Enter next grade (-1 to stop): "))

    # compute the average (each grade out of 100)
    average = total_grade / (count * 100)

    # print results
```

```
    print("Total of", count, "grades for ", name, "is", total_grade)
    print("Average grade for", name, "is", average * 100, "%")

    # read the next name
    name = input("Enter next student (or 'quit'): ")

print("Bye!")
```

Conclusion

Loops are used to repeat a set of instructions multiple times. First, we discussed while loops, which execute their body as long as their condition is true. Definite while loops use a counter variable to determine how many times the loop needs to run. Indefinite loops, on the other hand, compare their input to a sentinel value to determine when to stop running. Any definite while loop can also be written as a for loop, which runs for a range of values. We discussed the `range()` function to use a for loop in place of a definite while loop. We also discussed break and continue statements, and common situations that may cause an infinite loop.
